

DKIM Bulk Signature Verifier

User Guide & Options Reference

Version 1.7.0 | Build 2026-11-05 | verify_dkim_bulk.py

Bulk DKIM verification of .eml evidence files

Body-hash and cryptographic signature checks
Three public-key sourcing modes (live DNS or local key files)
DNS-retrieved keys preserved for offline re-verification
MD5 / SHA-256 hashing, per-file logs and CSV ledger
Optional regex keyword alerts on message bodies

Author: Jacques Boucher (jjrboucher@gmail.com)
https://github.com/jjrboucher/DKIM-Validator

Software: Apache License 2.0 | Documentation: CC BY 4.0 | Provided as-is, without warranty

Contents

1. Overview 3

2. Requirements and Installation 4

3. The Graphical Interface 5

4. Public Key Modes in Detail 7

5. Keyword Alerts 9

6. Command-Line Reference 10

7. How Verification Works 11

8. Output and Interpreting Results 12

9. Known Limitations and Notes (v1.7.0) 14

10. License, Warranty and Repository 15

Documentation licensed under CC BY 4.0

1. Overview

DKIM Bulk Signature Verifier is a Python tool that verifies the DomainKeys Identified Mail (DKIM) signature of every `.eml` file in a folder. For each message it recomputes the body hash, rebuilds the signed header block and checks the cryptographic signature against the signing domain's public key. Results are written to a single CSV ledger, supported by per-message log files, optional extracted artifacts and a folder of preserved public keys.

A valid DKIM signature shows that the signed headers and body have not been altered since the message was signed by the domain in the d= tag. In an investigation this helps establish whether an email produced as evidence is an unaltered copy of what the signing server sent.

What the tool does for each .eml file

- Calculates MD5 and SHA-256 hashes of the original file (evidence integrity).
- Optionally scans the message body against regex keyword patterns and records any matches.
- Reads the first `DKIM-Signature` header and extracts the domain (d=) and selector (s=).
- Locates the public key according to the selected key mode (live DNS or local key files).
- Preserves keys retrieved from DNS (Mode 1) in the `keys/` output folder.
- Canonicalises the body (simple or relaxed), computes the body hash and compares it with bh=.
- Rebuilds the signed header block from the h= list and verifies the b= signature.
- Records a PASS / FAIL / UNVERIFIED status in the CSV and writes a log file.

Supported algorithms and key formats

Item	Supported
Signature algorithms	rsa-sha256, rsa-sha1 (Ed25519 keys are parsed - see Limitations)
Canonicalisation	simple/simple, relaxed/relaxed and mixed combinations (c= tag); defaults to simple/simple
Body length limit	l= tag honoured (body truncated to the stated length before hashing)
Key formats	Raw DNS TXT record (v=DKIM1; k=rsa; p=...), zone-file style line, bare base64, PEM (BEGIN PUBLIC KEY / BEGIN RSA PUBLIC KEY), DER SubjectPublicKeyInfo, PKCS#1
Revoked keys	An empty p= tag is detected and reported as a revoked key

Documentation licensed under CC BY 4.0

What's new in v1.7.0

Change	Details
Key preservation (Mode 1)	Every key retrieved from live DNS is saved to `<Output>/keys/` as `selector._domainkey.domain.key` (Section 4.2)
Global key preservation (Mode 3)	A global key pasted into the GUI is saved to `keys/global_user_supplied.key` (Section 4.4)
Load Key File button	The Global Manual Key window can now load a key from a file
Mode 1 DNS lookups fixed	Corrected DNS-over-HTTPS URL; multi-string TXT records and CNAME answers handled correctly
Key_File_Path	Now populated for DNS keys and global keys
Log files	Now include Key File and Key SHA256 lines
License	Apache 2.0 notice added to the script header, GUI footer and --help

2. Requirements and Installation

Component	Purpose	Notes
Python 3	Runtime	Run with python / python3
cryptography	RSA / Ed25519 verification, key loading	Required. The script exits with an install hint if missing.
tkinter	Graphical interface	Bundled with most Windows/macOS Python installs; on Linux install python3-tk. Not needed with --no-gui.
Internet (HTTPS)	Mode 1 live DNS lookups	Only needed for Mode 1. Modes 2 and 3 work fully offline.

Get the tool and install the dependency:

```
git clone https://github.com/jjrboucher/DKIM-Validator.git
cd DKIM-Validator
pip install cryptography
```

Launching the tool

Launch method	Command	Result
Graphical (default)	python verify_dkim_bulk.py	Opens the configuration window
Command line	python verify_dkim_bulk.py --no-gui --src <folder> --out <folder> --mode <1|2|3>	Runs immediately with fixed defaults (see Section 6)
Help	python verify_dkim_bulk.py -h	Lists command-line options, version and license

Documentation licensed under CC BY 4.0

3. The Graphical Interface

The window is organised top-to-bottom in the order you configure a run. Each panel is described below.

3.1 Directory Workspace Configurations

Field	Default	Description
Source Folder	Current working directory	Folder containing the .eml files to verify. Use Browse... to pick a folder.
Output Folder	<current dir>/forensic_analysis_out	Where the CSV, logs, artifacts and keys/ folder are written. Created if it does not exist.

3.2 Directory Scan & Output Sub-Artifact Toggles

Option	Default	Effect
Scan Target Folder Recursively	On	Includes .eml files in all sub-folders. When off, only the top level of the Source Folder is scanned.
Extract & Save Canonicalized Body Artifacts (bodies/)	On	Saves the exact canonicalised body bytes that were hashed. Useful for showing a reviewer what was measured.
Extract & Save Cryptographic Header Blocks (signatures/)	On	Saves the reconstructed signed header block (the data the signature covers, with b= emptied).

3.3 Public Key Mapping Strategy

Select one of three modes that determine where the public key comes from (details in Section 4).

Mode	Label in GUI	Key source
1	Automated Live DNS Resolution Query (w/ Local Cascading Fallbacks)	Live DNS, then local key files. DNS keys saved to keys/
2	Local 1:1 Sidecar File Matching Pattern	<EmailName>.key next to each .eml
3	Local N:1 Mapping Model	selector._domainkey.domain.key, then optional global key

Configure Global Manual Key Fallback (Optional Mode 3) opens the *Supply Global Fallback Key* window. Either paste key material (any format in Section 1) or click **Load Key File...**, then click **Register Key Context**. A grey status line under the button confirms what is registered:

Status line	Meaning	At launch
Global key: none registered	No global key	-
Global key: pasted (will be saved to keys/global_user_supplied.key)	Key typed or pasted, or a loaded key that was edited	Saved to the keys/ folder
Global key: loaded from <file>	Key loaded from a file and not edited	Not copied; original file path recorded in the CSV

Documentation licensed under CC BY 4.0

3.4 Content Threat Keyword Alerts Configuration

Ticking **Enable Advanced Body Content Keyword Flagging Alerts** reveals an editable box pre-filled with the default patterns (Section 5) and a **Load Custom Keywords File (.txt)** button that appends patterns from a text file. One regular expression per line.

3.5 Launch

LAUNCH TRIAGE PROCESSING LOOP validates the settings, closes the window and starts processing. Progress is shown in the console (e.g. `Analyzing items: [3/40]`). Before launching, the tool:

- Compiles every keyword line; a broken expression stops the launch and reports the offending line number.
- In Mode 3, warns if no global key is registered and no `*_domainkey*.key` files are found in the top level of the Source Folder, and asks whether to proceed.

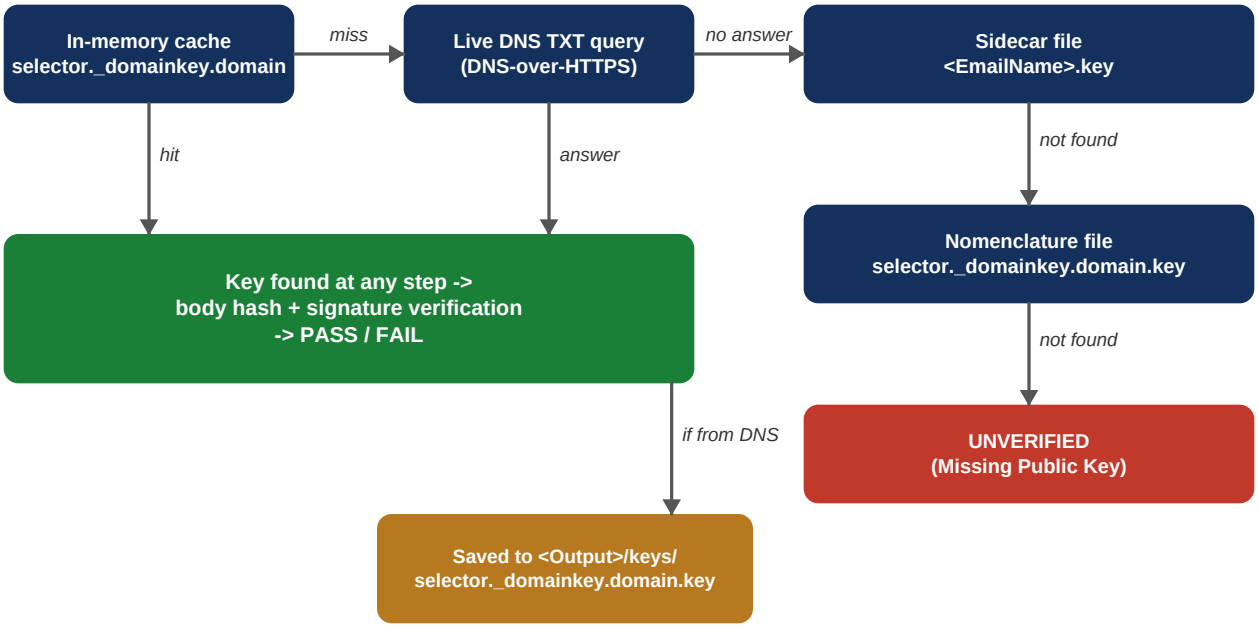
Documentation licensed under CC BY 4.0

4. Public Key Modes in Detail

DKIM keys are published in DNS at `<selector>._domainkey.<domain>`. Because domains rotate or retire keys, the key that signed an old email may no longer be in DNS. The three modes let you choose between the live record and keys you have preserved locally.

4.1 Mode 1 - Live DNS with local fallbacks

Mode 1 - key resolution cascade (per .eml file)



- Retrieved keys are cached in memory for the run, so each selector/domain pair is queried only once.
- Lookups are sent over HTTPS to Google Public DNS (`dns.google/resolve`). CNAME answers are followed and long records split into several strings are joined. Consider whether sending the domains/selectors under investigation to a third party is acceptable for your case.
- The CSV column `Key_Source_Used` records which step supplied the key.

4.2 Preserving keys retrieved from DNS

> **New in v1.7.0**
> Every key successfully retrieved from DNS is written to `<Output Folder>/keys/` using the nomenclature `<selector>._domainkey.<domain>.key`.

Behaviour	Detail
File content	The TXT record exactly as used for verification (e.g. v=DKIM1; k=rsa; p=...), with no added newline. The file's SHA-256 therefore equals Key_SHA256 in the CSV.
When saved	Only for keys obtained from live DNS. Keys from local fallback files are not copied.
Key_File_Path	Points to the saved file for LIVE_DNS_QUERY and CACHE_MEMORY rows.
Same key already saved	If the output folder already holds an identical file, it is reused.

Documentation licensed under CC BY 4.0

Behaviour	Detail
Different key already saved	Existing files are never overwritten. The new key is saved as selector._domainkey.domain.<UTCstamp>.key, which shows the record changed between runs.
Unsafe characters	Selector/domain values come from the email header. Characters other than letters, digits, '.', '_' and '-' are replaced with '_' so a crafted header cannot write outside keys/.

Re-verify offline with Mode 3
Copy the files from keys/ into the folder holding the .eml files and re-run in Mode 3. The result is repeatable and independent of later key rotation. Record retrieval time from the log or file timestamps if it matters for your report.

4.3 Mode 2 - 1:1 sidecar files (offline)

Each email must have a key file with the same base name in the same folder. Only that file is used.

```
evidence/
  Invoice_2026-03-14.eml
  Invoice_2026-03-14.key     <- key for this email only
```

4.4 Mode 3 - N:1 nomenclature files (offline)

One key file per selector/domain, shared by every email signed with that key, stored in the same folder as the emails. If no matching file exists, the Global Manual Key (if registered) is used.

```
evidence/
  msg001.eml   (d=example.com; s=selector1)
  msg002.eml   (d=example.com; s=selector1)
  selector1._domainkey.example.com.key   <- used for both
```

New in v1.7.0
A global key **pasted** into the GUI is saved to `<Output Folder>/keys/global_user_supplied.key` at launch, and that path appears in Key_File_Path. A global key **loaded from a file** (and not edited) is not copied; its original path is recorded instead. The same no-overwrite rule as Section 4.2 applies.

Documentation licensed under CC BY 4.0

Key_Source_Used values

Value	Meaning	Key_File_Path
CACHE_MEMORY	Mode 1 - key reused from an earlier lookup in this run	keys/ file
LIVE_DNS_QUERY	Mode 1 - key obtained from DNS	keys/ file
FALLBACK_SIDECAR_FILE	Mode 1 - DNS failed; <EmailName>.key used	Sidecar file
FALLBACK_NOMENCLATURE_FILE	Mode 1 - DNS failed; selector._domainkey.domain.key used	Nomenclature file
SIDECAR_FILE	Mode 2 - <EmailName>.key used	Sidecar file
N1_NOMENCLATURE_FILE	Mode 3 - selector._domainkey.domain.key used	Nomenclature file
GLOBAL_USER_SUPPLIED_KEY	Mode 3 - Global Manual Key used	keys/ file (pasted) or original file (loaded)
UNKNOWN	No key located (status is UNVERIFIED) or signature absent	N/A

5. Keyword Alerts

Keyword alerts flag messages whose body matches investigator-defined regular expressions. They do not affect the DKIM result. Matching patterns are listed in the `Threat_Alerts` CSV column (or `CLEAN` if none match).

Default patterns

Theme	Pattern (case-insensitive)
Payment / banking	wire transfer | swift | routing number | bank account | invoice
Cryptocurrency	bitcoin | crypto | usdt | wallet phrase | private key
Credential phishing	password reset | verify your account | login expired | microsoft team
Legal / sensitive	subpoena | warrant | investigation | court notice | classified

Writing custom patterns

- One Python regular expression per line; blank lines are ignored.
- Prefix with `(?i)` for case-insensitive matching.
- Custom files are appended to whatever is already in the box - delete the defaults first if you do not want them.

```
(?i)(urgent payment|change of bank details)
(?i)\b(iban|bic)\b
(?i)gift ?cards?
```

Documentation licensed under CC BY 4.0

> **Important: the scan runs on the raw body**
> Patterns are matched against the undecoded MIME body. Text inside base64 or quoted-printable encoded parts and attachments will usually not match. Treat CLEAN as 'no match in raw text', not as proof the content is benign.

6. Command-Line Reference

Argument	Required with --no-gui	Description
--no-gui	-	Skip the window and run from the command line
--src <folder>	Yes	Source folder containing .eml files
--out <folder>	Yes	Output folder for the CSV, logs, artifacts and keys/
--mode {1,2,3}	Yes	Public key mode (Section 4)
-h, --help	-	Show help, version, repository and license

Fixed settings in command-line mode

Setting	CLI behaviour
Recursive scan	Always on
Save bodies/ and signatures/	Always on
Save DNS keys to keys/ (Mode 1)	Always on
Keyword alerts	Off (Threat_Alerts = CLEAN)
Global manual key (Mode 3)	Not available - provide nomenclature .key files instead

Examples

```
# Live DNS with local fallbacks (retrieved keys saved to C:\Case123\dkim\keys)
python verify_dkim_bulk.py --no-gui --src C:\Case123\eml --out C:\Case123\dkim --mode 1

# Fully offline, one shared key file per selector/domain
python verify_dkim_bulk.py --no-gui --src ./eml --out ./dkim_out --mode 3
```

Documentation licensed under CC BY 4.0

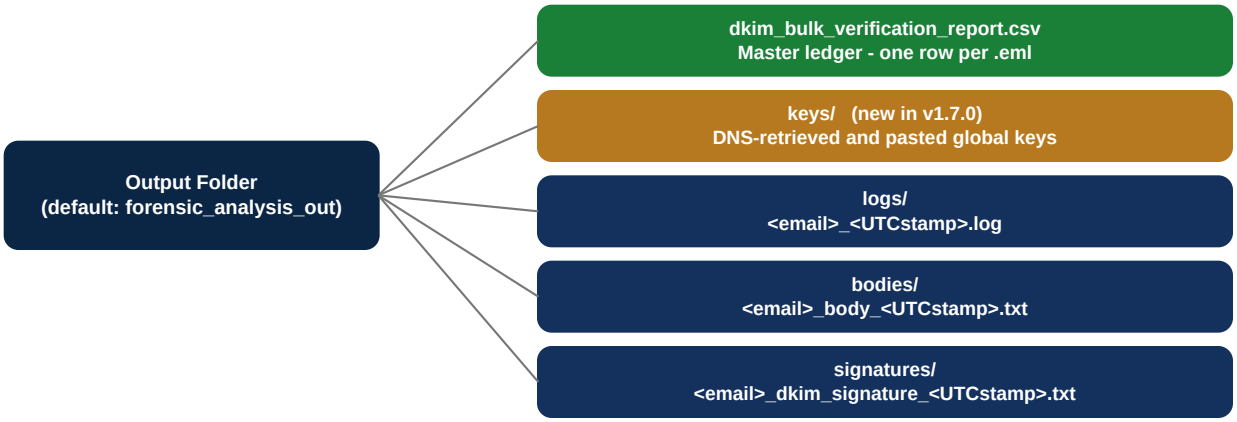
7. How Verification Works

Step	Operation	Detail
1	Hash the file	MD5 and SHA-256 of the unmodified .eml bytes
2	Split headers / body	At the first blank line (CRLF CRLF, or LF LF as a fallback)
3	Parse signature	First DKIM-Signature header; tags a, c, d, s, h, bh, b, l
4	Get public key	According to the selected mode; DNS keys saved to keys/
5	Canonicalise body	simple or relaxed per c=; truncated to l= if present
6	Body hash	SHA-256 (or SHA-1 for rsa-sha1), base64; compared with bh=
7	Rebuild signed block	Headers in h= order (bottom-up for repeated names), then the DKIM-Signature with b= emptied
8	Verify signature	RSA PKCS#1 v1.5 with SHA-256/SHA-1 over the signed block
9	Record	CSV row, log file, optional bodies/ and signatures/ artifacts

Documentation licensed under CC BY 4.0

8. Output and Interpreting Results

Output folder layout



- Timestamps are UTC in the form YYYYMMDDTHHMMSSZ, so repeated runs do not overwrite logs or artifacts.
- The CSV is overwritten on each run to the same Output Folder - use a new folder per run to keep history.
- Log, bodies and signatures files are written only when verification reaches the signature check (not for UNVERIFIED or runtime-error files).
- keys/ is created only when a key is saved (Mode 1 DNS key, or a pasted Mode 3 global key).

CSV columns

Column	Content
EML_Filename	File name of the message
MD5_Hash / SHA256_Hash	Hashes of the original .eml file
DKIM_Domain / DKIM_Selector	d= and s= tags (N/A if unsigned)
Key_Source_Used	Where the key came from (Section 4)
Key_File_Path	Path of the key file used - including keys/ files for DNS and pasted global keys - or N/A
Key_SHA256	SHA-256 of the key text used - documents exactly which key was applied
Body_Hash_Result	MATCH / MISMATCH / ERROR
Signature_Result	VERIFIED / FAILED / ERROR
Overall_Status	See status table below
Threat_Alerts	Matching keyword patterns, or CLEAN

Log file fields

Each log records tool, version, build, author, target file, SHA-256, overall result, body hash, signature result, keyword alerts and key route, plus (new in v1.7.0) **Key File** and **Key SHA256**.

Documentation licensed under CC BY 4.0

Overall_Status values

Status	Meaning
PASS	Body hash matches AND signature verified
FAIL	Body hash mismatch and/or signature failed
UNVERIFIED (No Signature)	Message has no DKIM-Signature header
UNVERIFIED (Missing Public Key)	No key found for the selected mode
RUNTIME_ERROR (...)	Processing error, e.g. malformed file, revoked/invalid key; message included

Reading a result

Body hash	Signature	Likely interpretation
MATCH	VERIFIED	Signed headers and body unchanged since signing (PASS)
MISMATCH	VERIFIED	Signed headers intact but body changed - e.g. edited content, or altered in transit by a gateway/disclaimer/footer
MATCH	FAILED	Body intact but a signed header changed, wrong/rotated key, or export altered headers
MISMATCH	FAILED	Message modified, wrong key, or file not a faithful copy (e.g. line-ending conversion)

A FAIL is not automatically proof of tampering
Legitimate causes include mailing-list or security-gateway modification, key rotation (Mode 1 retrieves today's key, not the one in use when the email was sent) and exports that re-encode the message. Corroborate with headers, server logs and other copies of the message.

Documentation licensed under CC BY 4.0

9. Known Limitations and Notes (v1.7.0)

Area	Observation	Impact / workaround
Multiple signatures	Only the first DKIM-Signature header is verified.	Messages with several signatures (e.g. ESP plus own domain) report on the top one only.
Line endings	Header splitting and body canonicalisation assume CRLF.	LF-only .eml files (common from some exports/Linux tools) will likely FAIL or error. Convert to CRLF on a working copy.
Ed25519	Ed25519 keys are parsed, but the signature is checked over the raw header block rather than its SHA-256 digest (RFC 8463).	ed25519-sha256 signatures are likely to report FAILED.
Tags not evaluated	x= (expiry), t= (timestamp) and i= (identity) are not checked.	Review these manually from the signature header if relevant.
DNS errors	Network or resolver errors are not reported; the lookup is treated as 'no answer'.	If you expect DNS keys but see fallbacks or UNVERIFIED, check connectivity to dns.google.
Keyword scan	Runs on the raw, undecoded body.	Encoded content will not match (Section 5).
Mode 3 pre-check	The GUI warning looks only at the top level of the Source Folder.	Nomenclature files must sit beside the emails they apply to.
CLI global key	No command-line option for a global key.	Use nomenclature .key files in CLI runs.

Good practice

- Work on copies of evidence; record the SHA-256 values in the CSV against your intake hashes.
- Keep the keys/ folder with the case file - it is the record of which public keys were used.
- Prefer Mode 3 with preserved keys for repeatable results.
- Use a dedicated output folder per run and keep the CSV, logs, artifacts and keys together.

Documentation licensed under CC BY 4.0

10. License, Warranty and Repository

Repository

The script, this guide and the README are published at https://github.com/jjrboucher/DKIM-Validator. Check the repository for the latest version, release notes and issue reporting.

Licensing

Component	License	Summary
Software (verify_dkim_bulk.py)	Apache License 2.0	Free to use, modify and distribute, including commercially. Keep the copyright and license notices, state significant changes, and include a copy of the license. Includes an express patent grant. The notice is in the comment header at the top of the script.
Documentation (this guide, README)	Creative Commons Attribution 4.0 International (CC BY 4.0)	Free to share and adapt for any purpose, provided appropriate credit is given to the author, a link to the license is included and any changes are indicated.

Full license texts: apache.org/licenses/LICENSE-2.0 and creativecommons.org/licenses/by/4.0.

> **Disclaimer of warranty**
> This tool is provided free of charge, "AS IS", WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied, including any warranties of merchantability, fitness for a particular purpose, accuracy or non-infringement. In no event shall the author be liable for any claim, damages or other liability arising from the use of the tool or its results. Users are responsible for validating results and for the evidential weight they place on them.

Suggested attribution for this guide

DKIM Bulk Signature Verifier - User Guide (v1.7.0) by Jacques Boucher, licensed under CC BY 4.0. https://github.com/jjrboucher/DKIM-Validator

Tool	DKIM Bulk Signature Verifier
Version	1.7.0
Build date	2026-11-05
Script	verify_dkim_bulk.py
Author	Jacques Boucher (jjrboucher@gmail.com)
Repository	https://github.com/jjrboucher/DKIM-Validator
Licenses	Software: Apache-2.0 | Documentation: CC BY 4.0